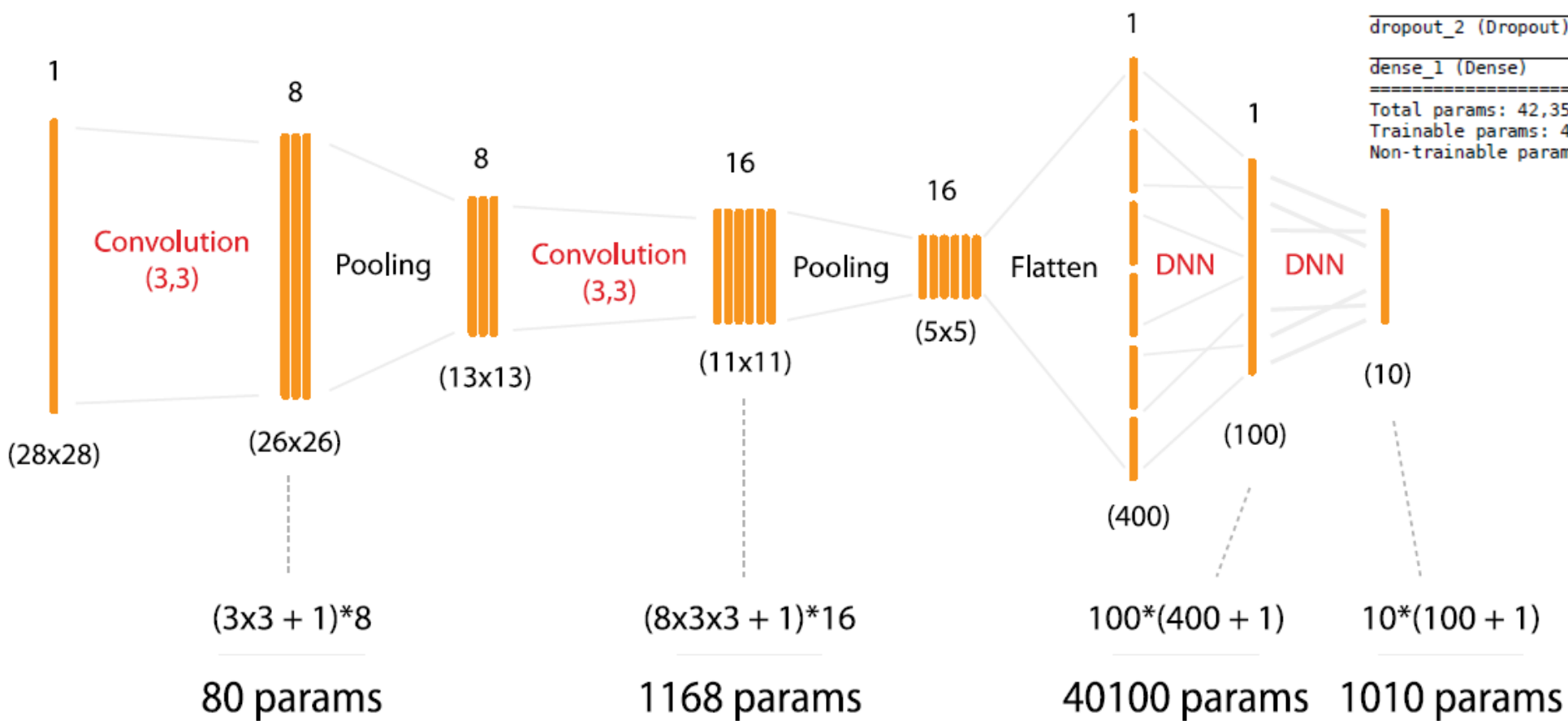


Previously ...

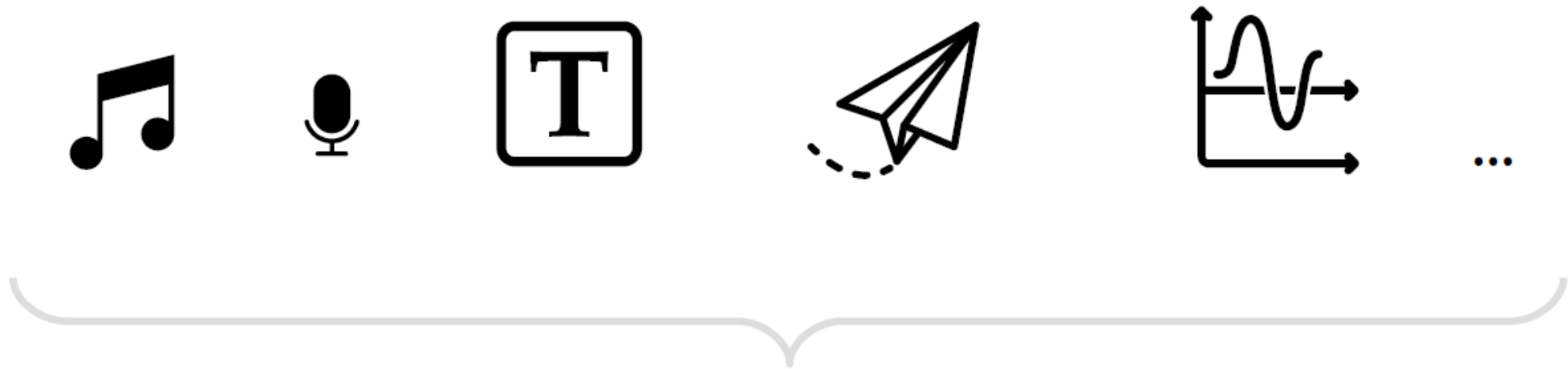


Model: "sequential"

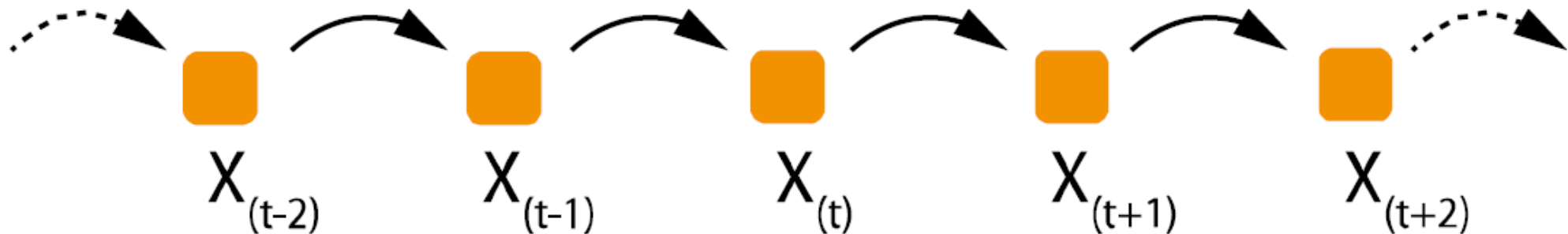
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 8)	80
max_pooling2d (MaxPooling2D)	(None, 13, 13, 8)	0
dropout (Dropout)	(None, 13, 13, 8)	0
conv2d_1 (Conv2D)	(None, 11, 11, 16)	1168
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 16)	0
dropout_1 (Dropout)	(None, 5, 5, 16)	0
flatten (Flatten)	(None, 400)	0
dense (Dense)	(None, 100)	40100
dropout_2 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 10)	1010

Total params: 42,358
Trainable params: 42,358
Non-trainable params: 0

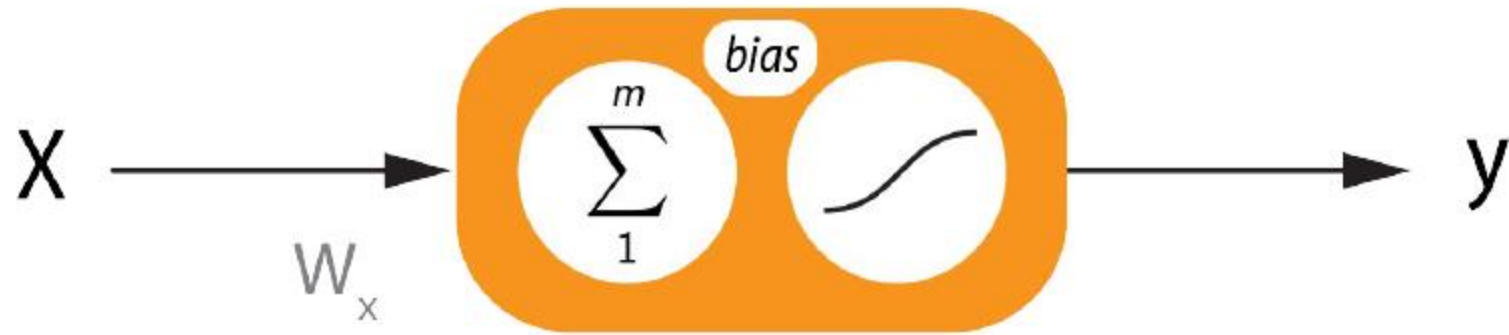
❑ Modèle séquentielle



❑ Une information qui change d'état dans le temps



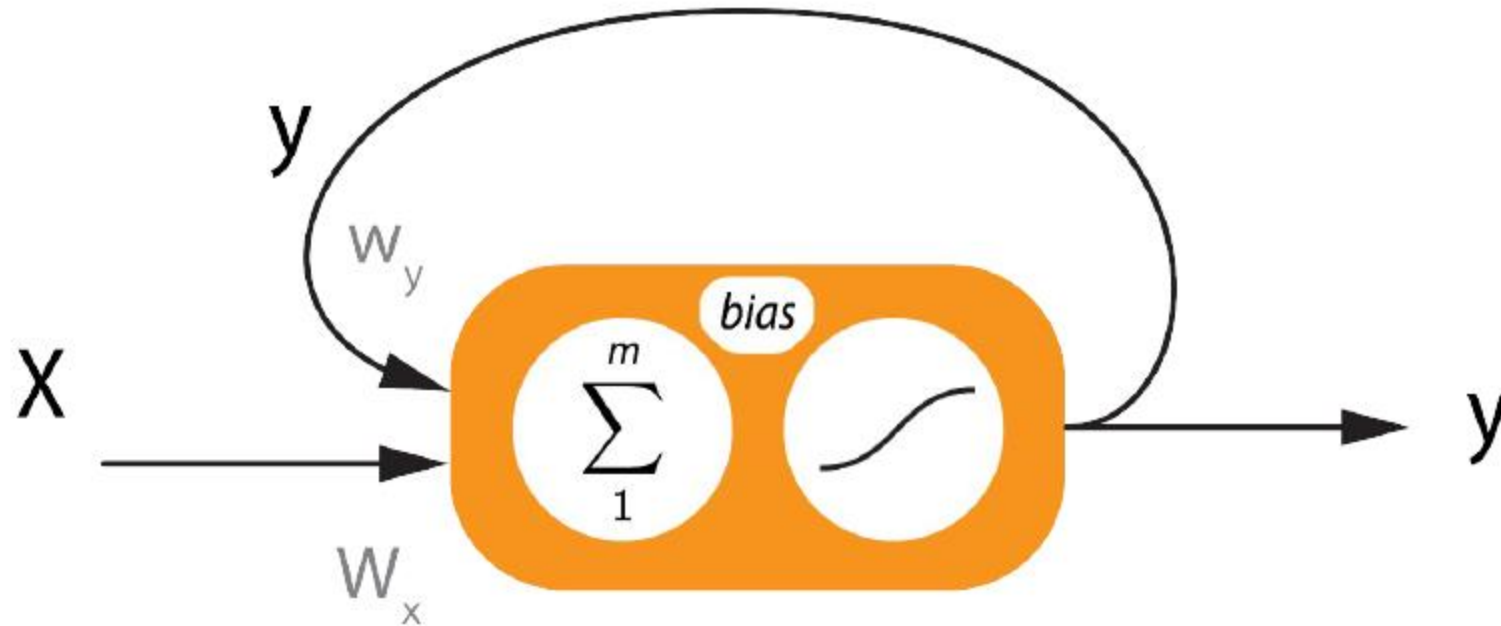
❑ Neurone classique



$$y = \sigma(W_x^T \cdot X + b)$$

□ Neurone récurrent

✓ Boucler sur le résultat de sortie



$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

Where :

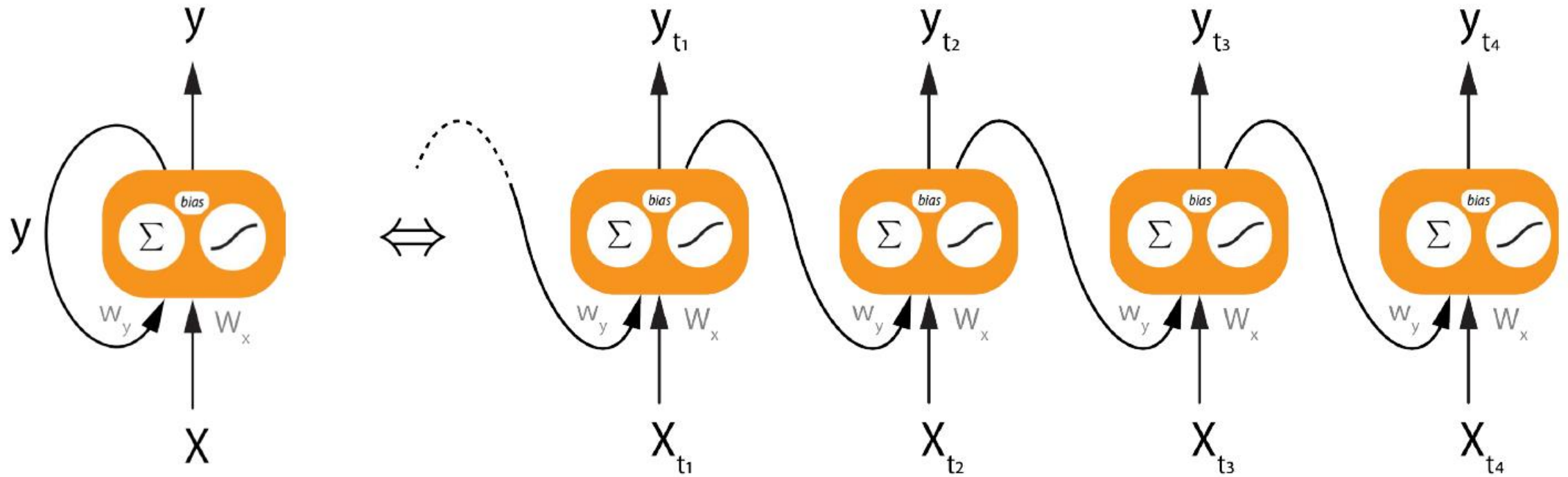
w_y : is a scalar

W_x : is a vector

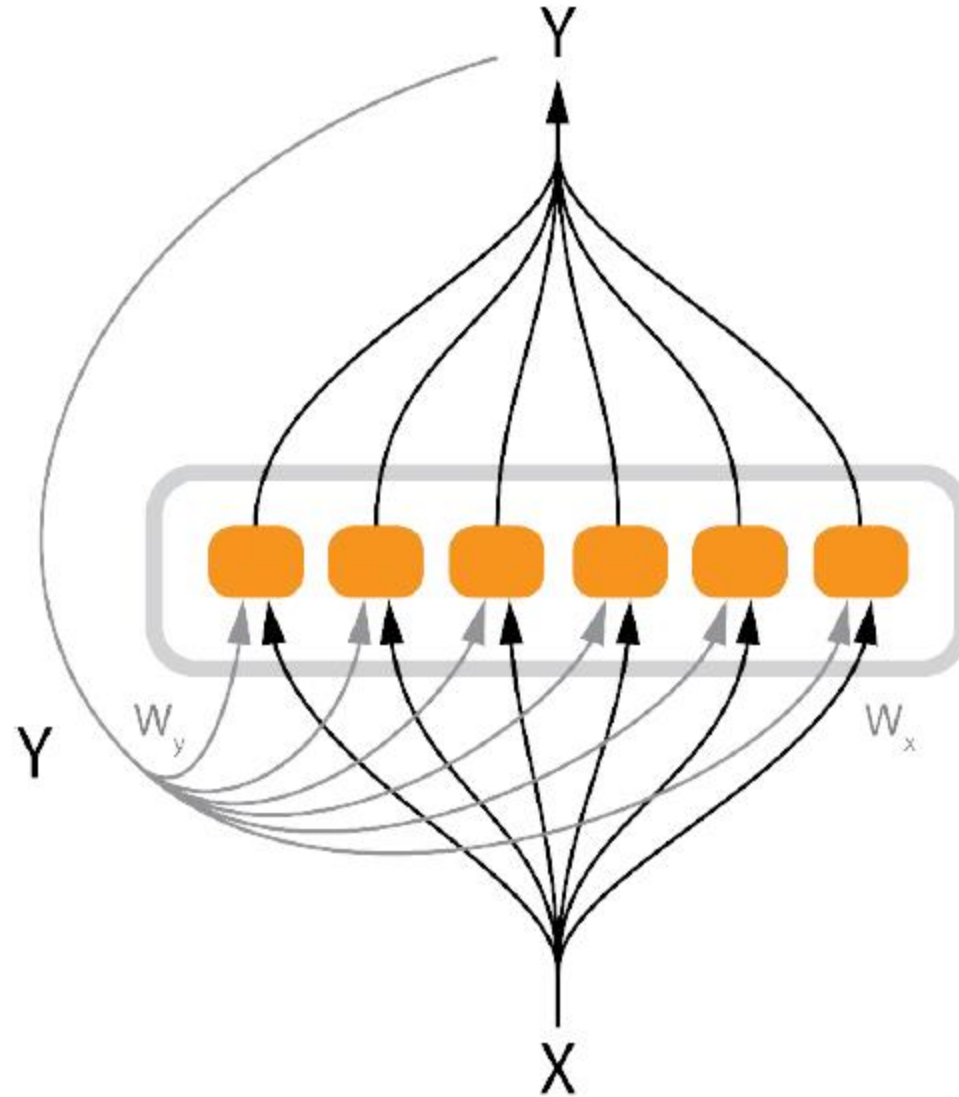
b : is a scalar

y : is a scalar

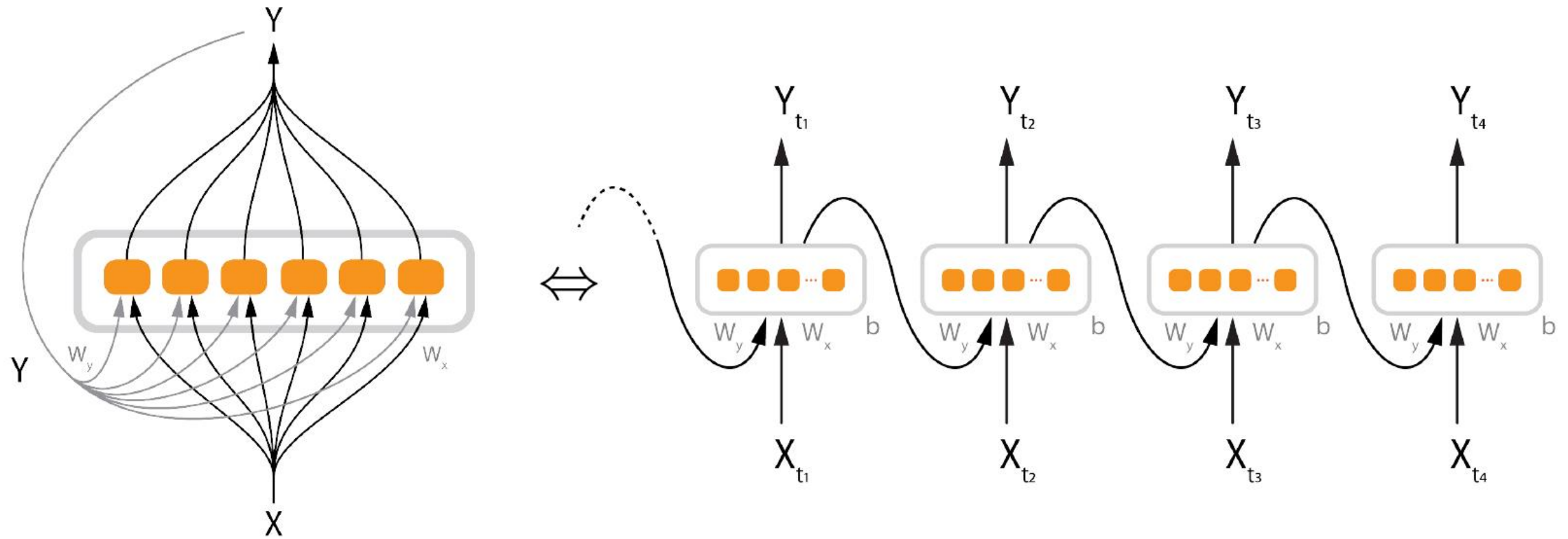
□ Couche de neurones récurrents



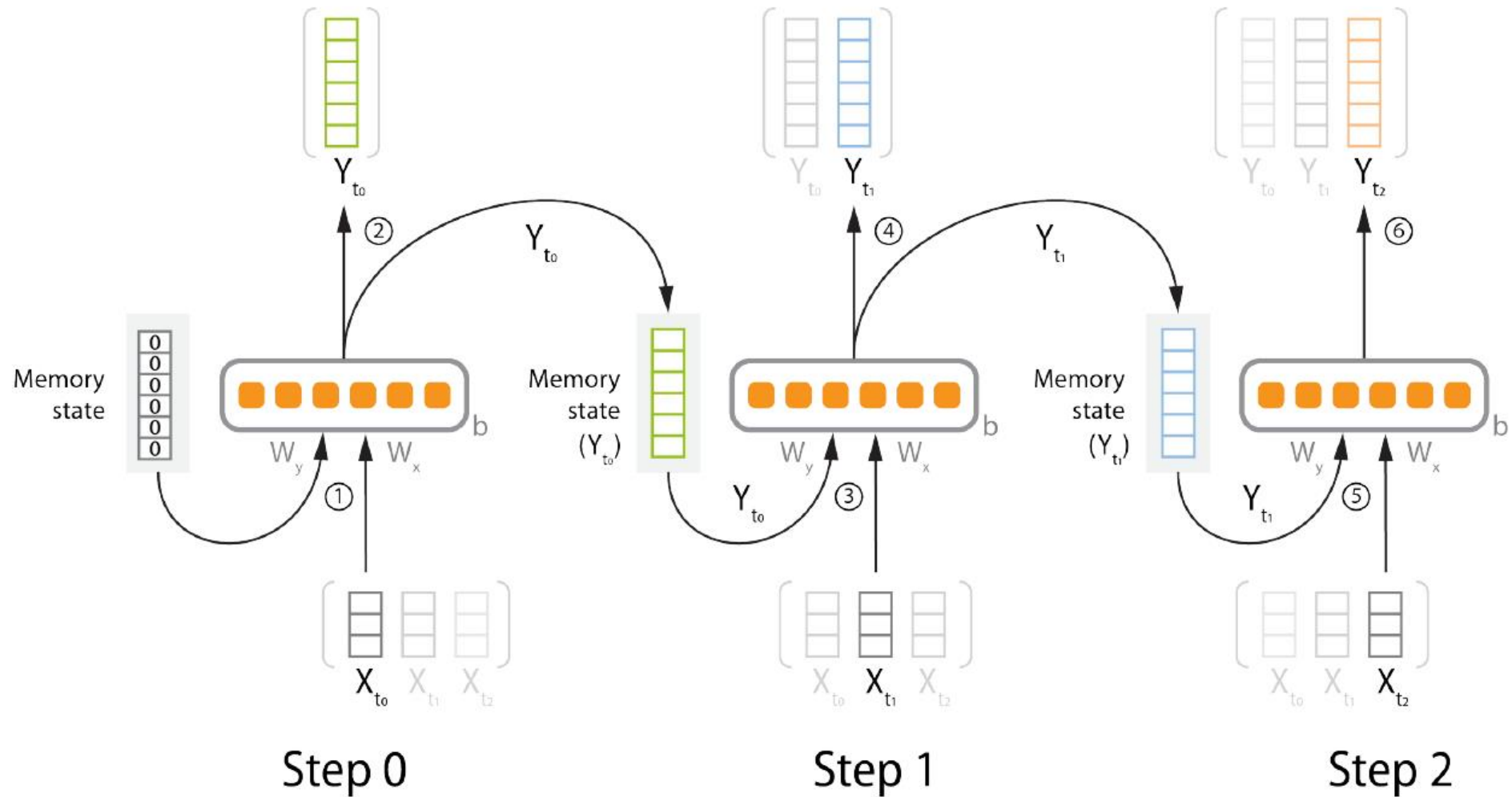
❑ Couche de neurones récurrents



❑ Fonctionnement d'une couche de neurones récurrents



❑ Fonctionnement d'une couche de neurones récurrents



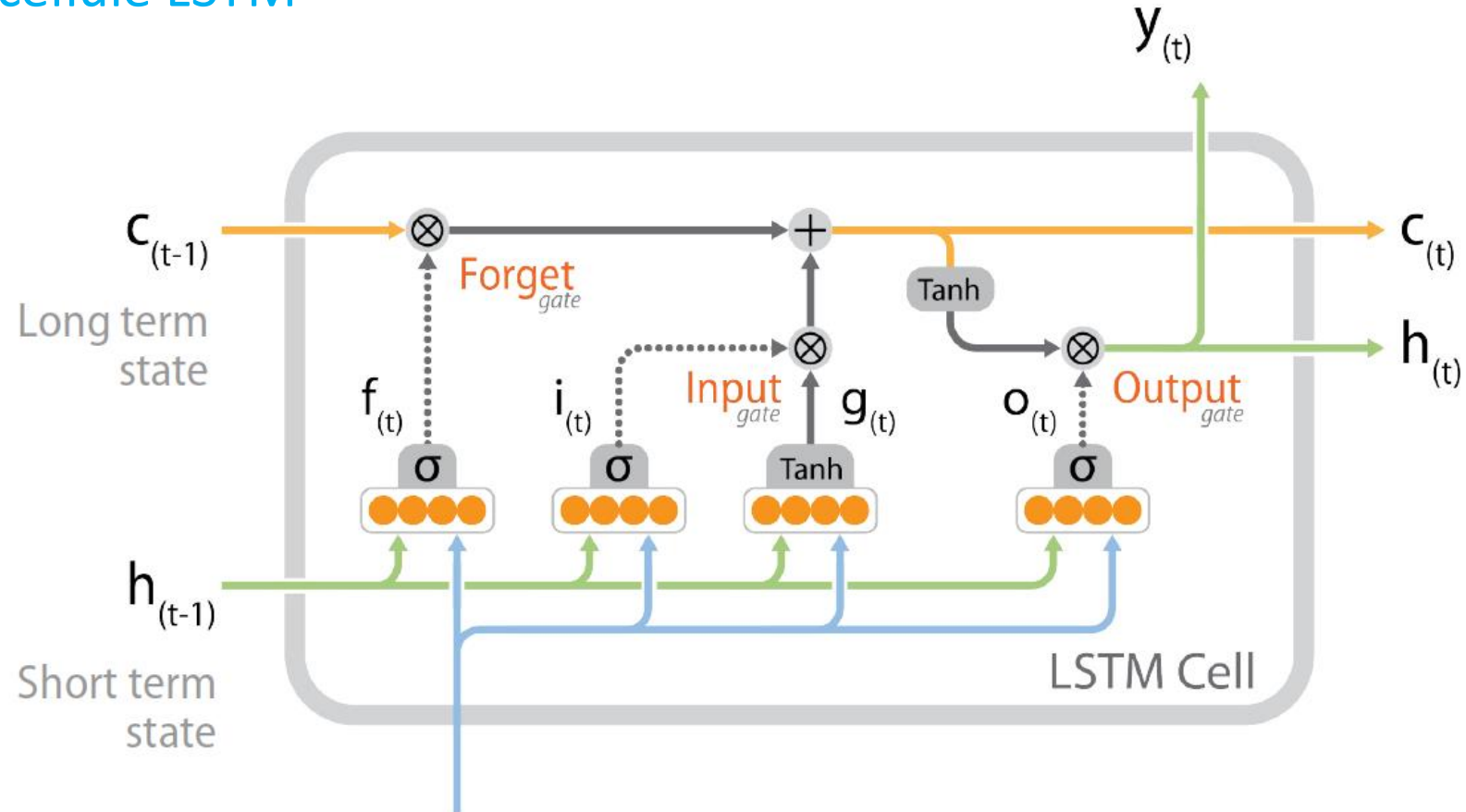
$$Y_{(t)} = \phi (W_x^T \cdot X_{(t)} + W_y^T \cdot Y_{(t-1)} + b)$$

❑ A long terme : dissolution de la cellule de récurrence

- Vanishing
- Exploding Gradient



❑ Utiliser la cellule LSTM



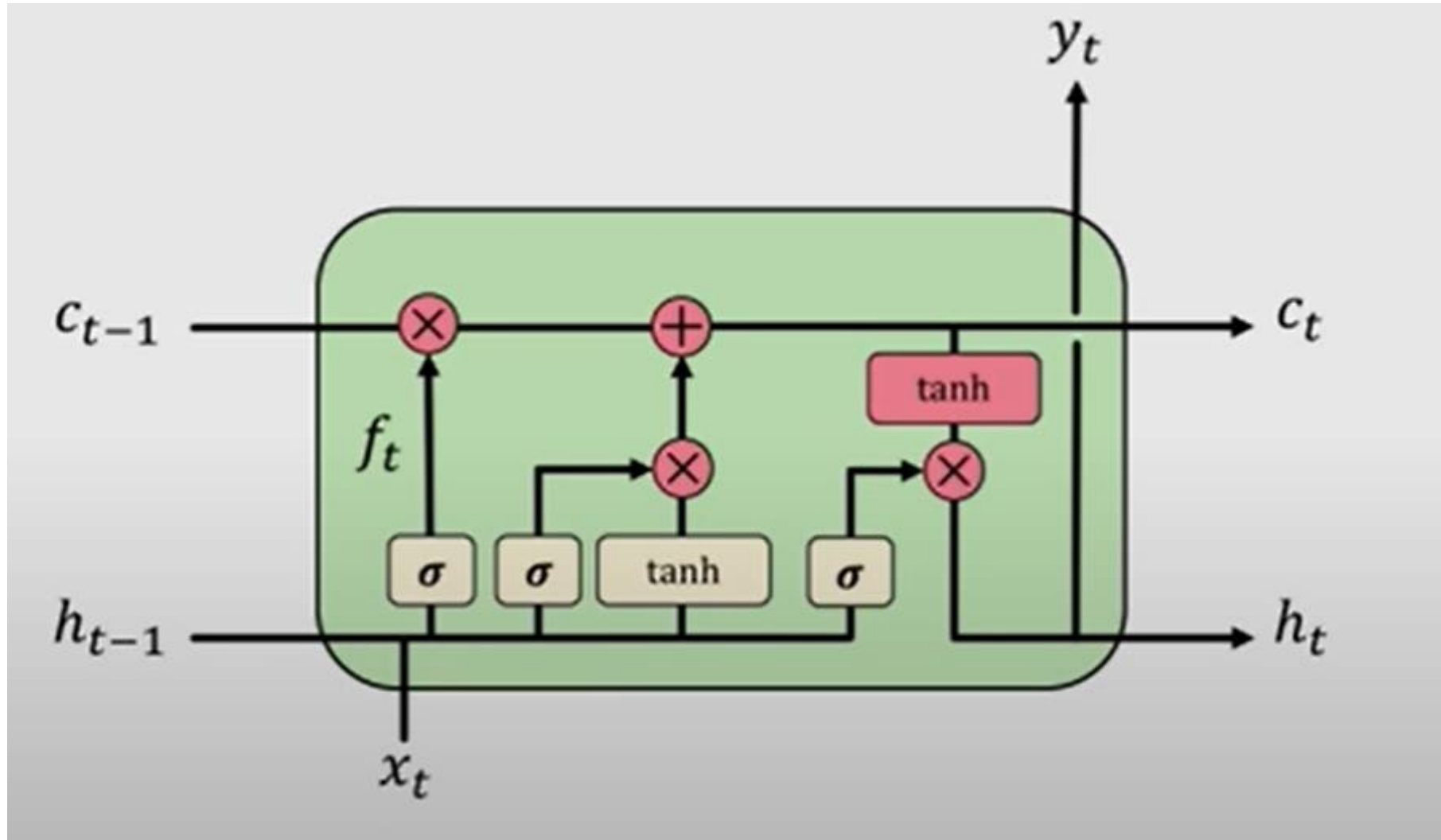
❑ Principe de fonctionnement : reverse engineering

1) Forget

2) Store

3) Update

4) Outcome



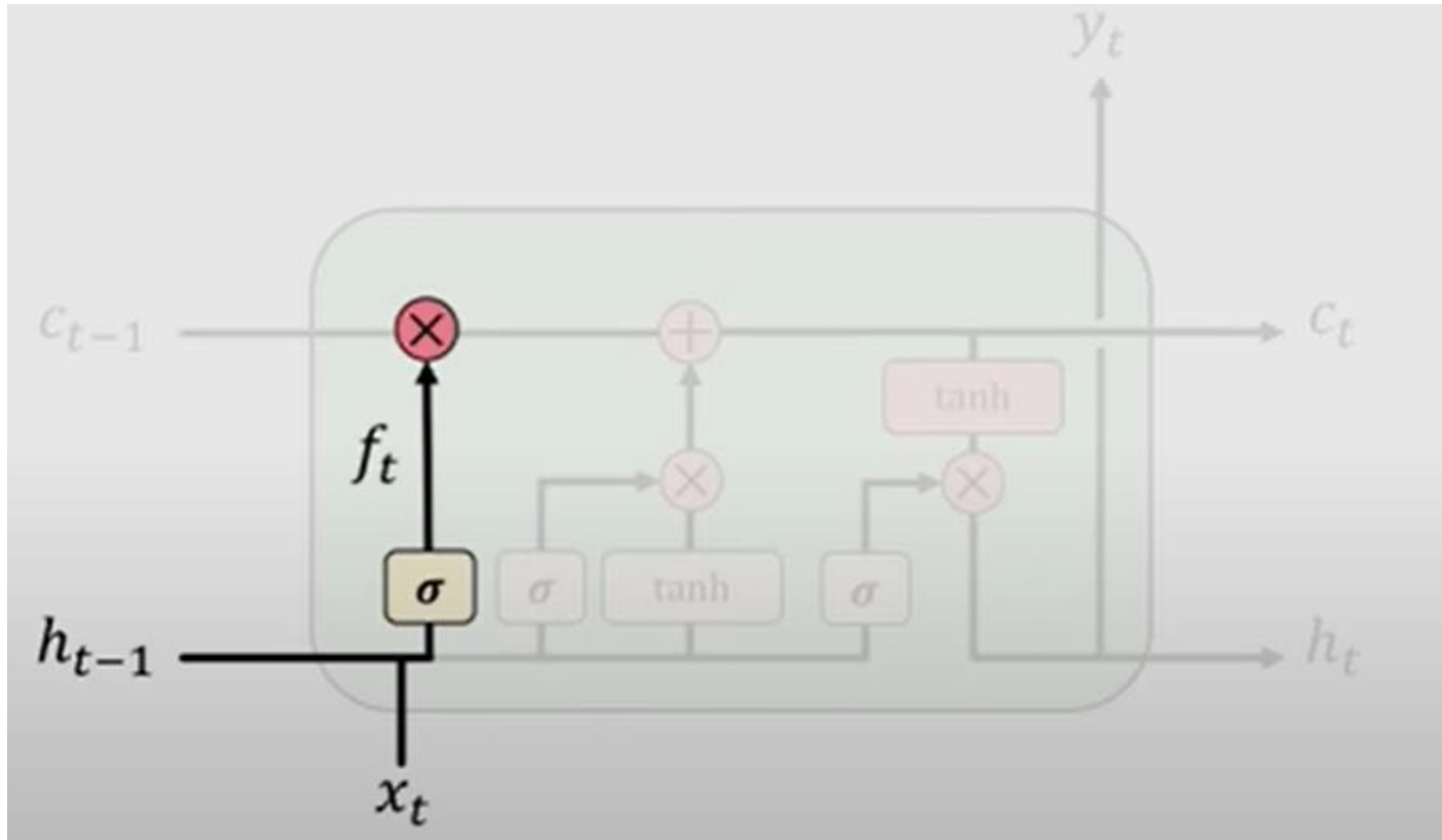
❑ Principe de fonctionnement : reverse engineering

1) Forget

2) Store

3) Update

4) Outcome



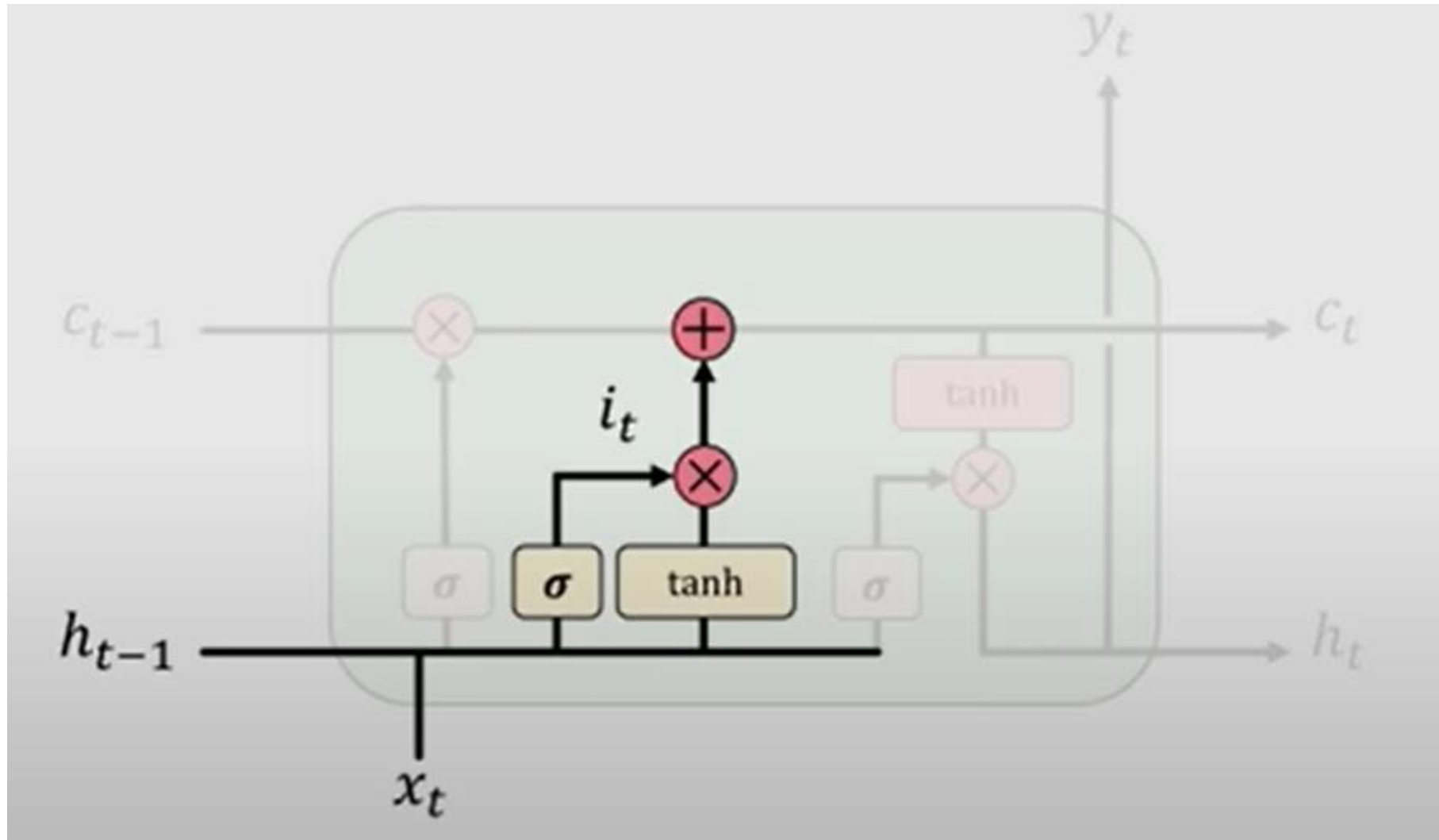
❑ Principe de fonctionnement : reverse engineering

1) Forget

2) Store

3) Update

4) Outcome



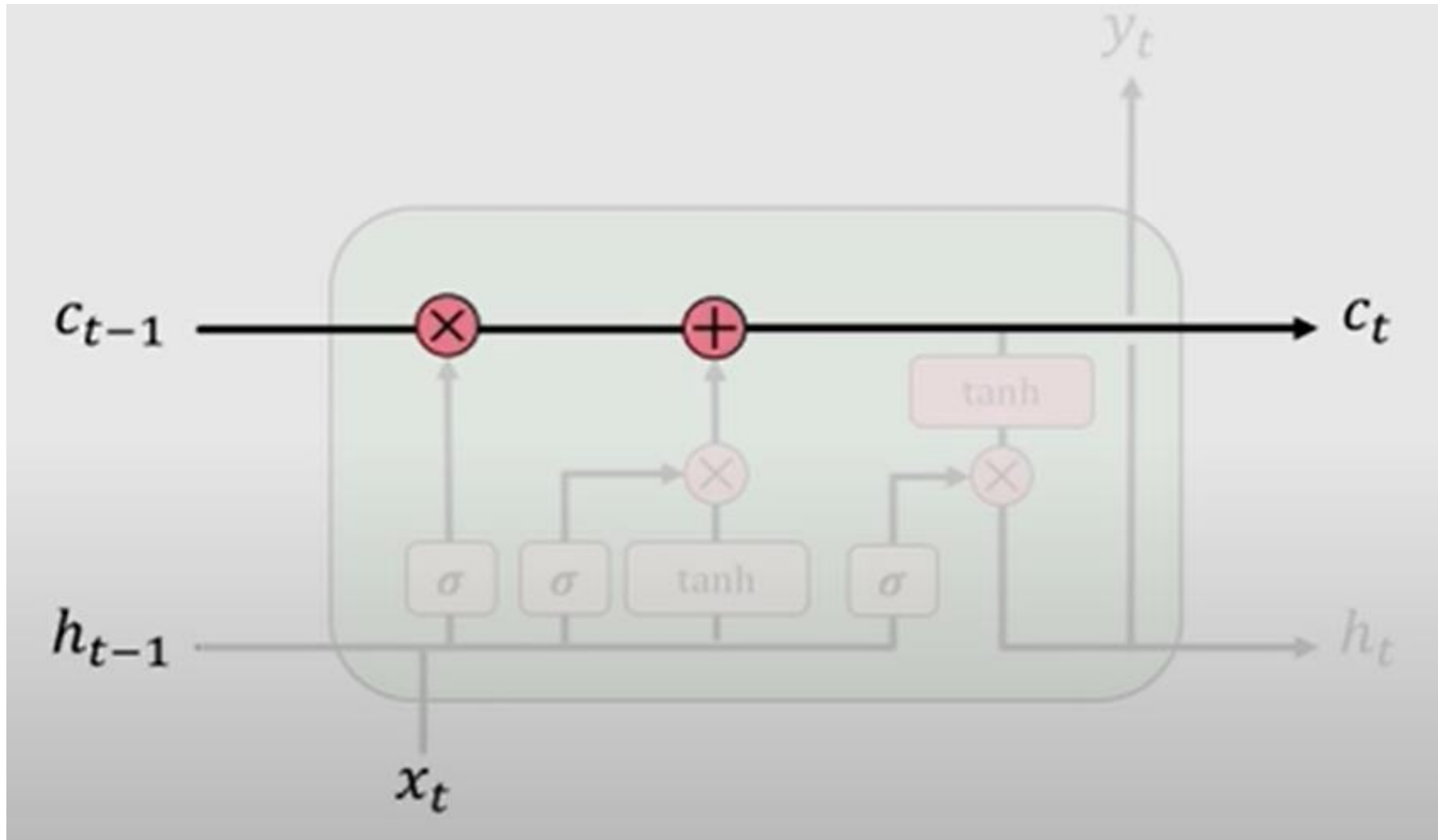
❑ Principe de fonctionnement : reverse engineering

1) Forget

2) Store

3) Update

4) Outcome



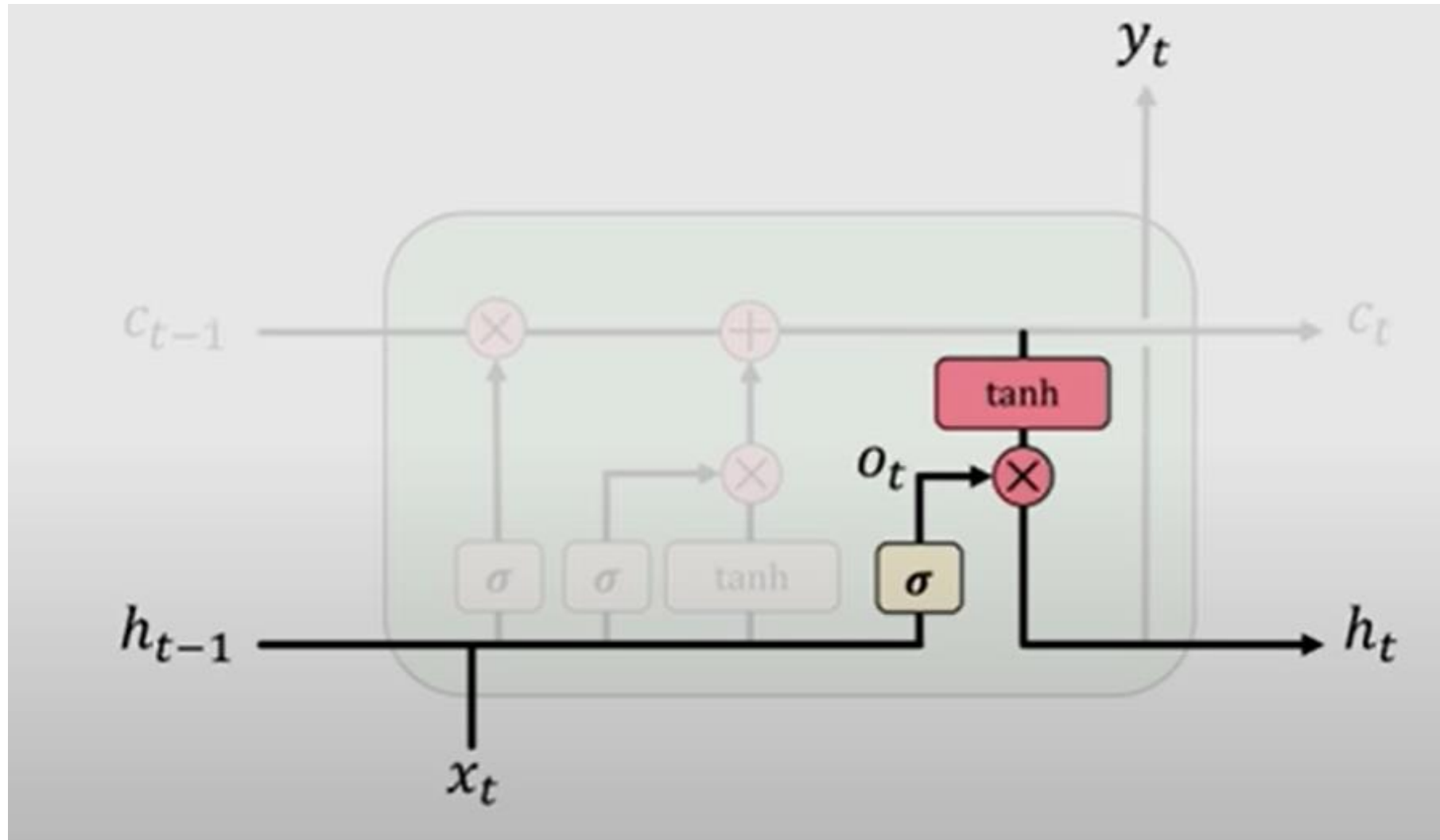
❑ Principe de fonctionnement : reverse engineering

1) Forget

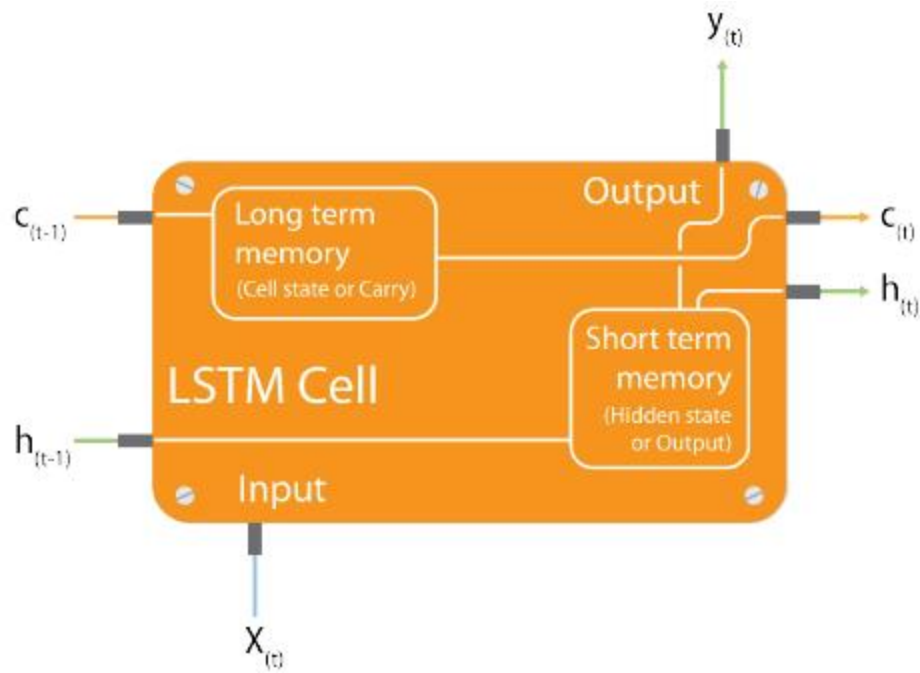
2) Store

3) Update

4) **Output**



❑ LSTM with Keras Tensorflow



```
inputs = tf.random.normal([32, 20, 8])
```

```
lstm = tf.keras.layers.LSTM(16)  
output = lstm(inputs)
```

Series to **vector**

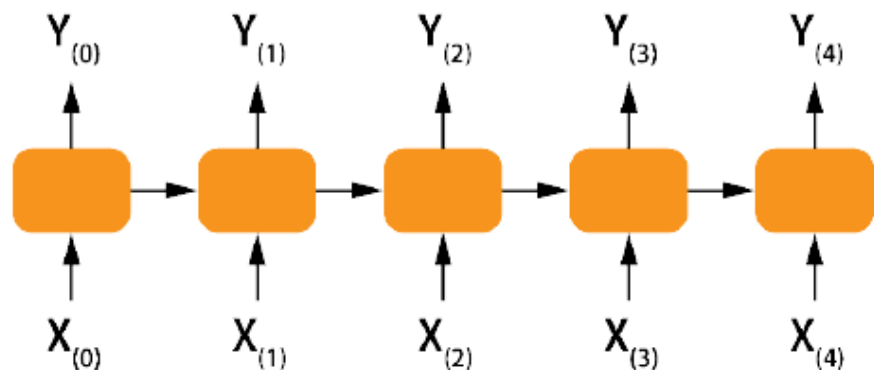
Inputs shape is : (32, 20, 8)
Output shape is : (32, 16)

```
lstm = tf.keras.layers.LSTM(18, return_sequences=True, return_state=True)  
output, memory_state, carry_state = lstm(inputs)
```

Series to **serie**

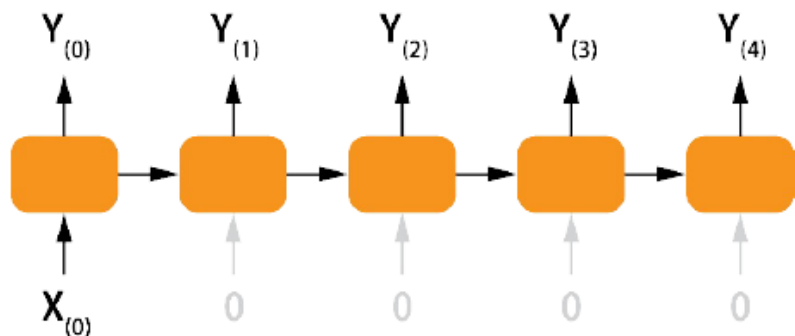
Output shape : (32, 20, 18)
Memory state : (32, 18)
Carry state : (32, 18)

□ Domaines d'applications



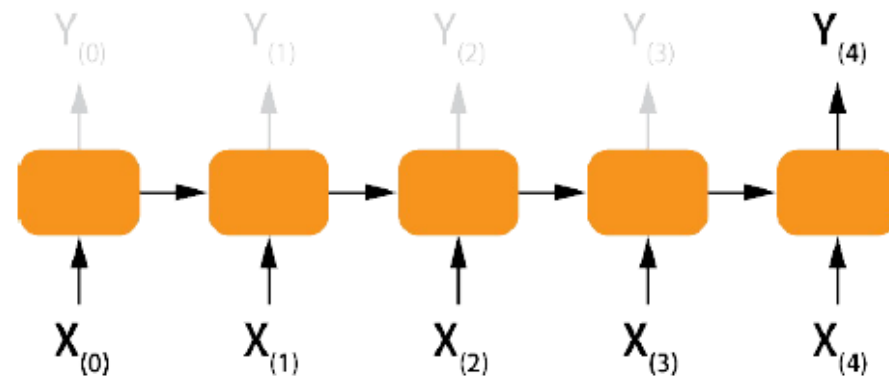
Serie to serie

- Prédiction série temporelle



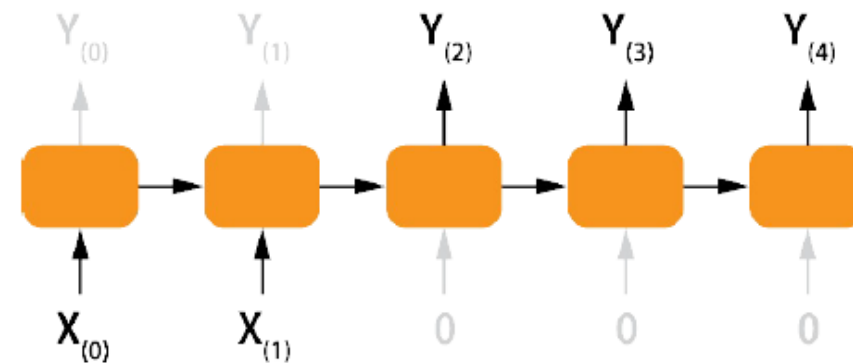
Vector to serie

- Annotation d'images



Serie to vector

- Analyse de sentiments



Encoder-decoder

- Traduction de textes

□ How to predict a sequence (time's serie)

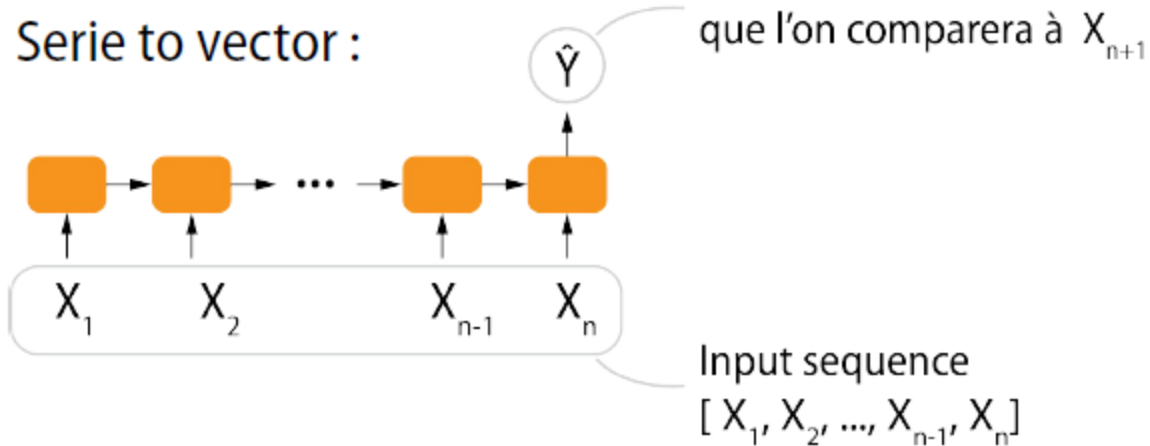
Known sequence : $[X_1, X_2, \dots, X_n, X_{n+1}]$

Input Sequence : $X = [X_1, X_2, \dots, X_n]$

Expected output : $Y = [X_{n+1}]$

The objective will be to train our RNN network to predict the $n+1$ vector of our input sequence :

Serie to vector :



Ordered dataset

